

Compiladores

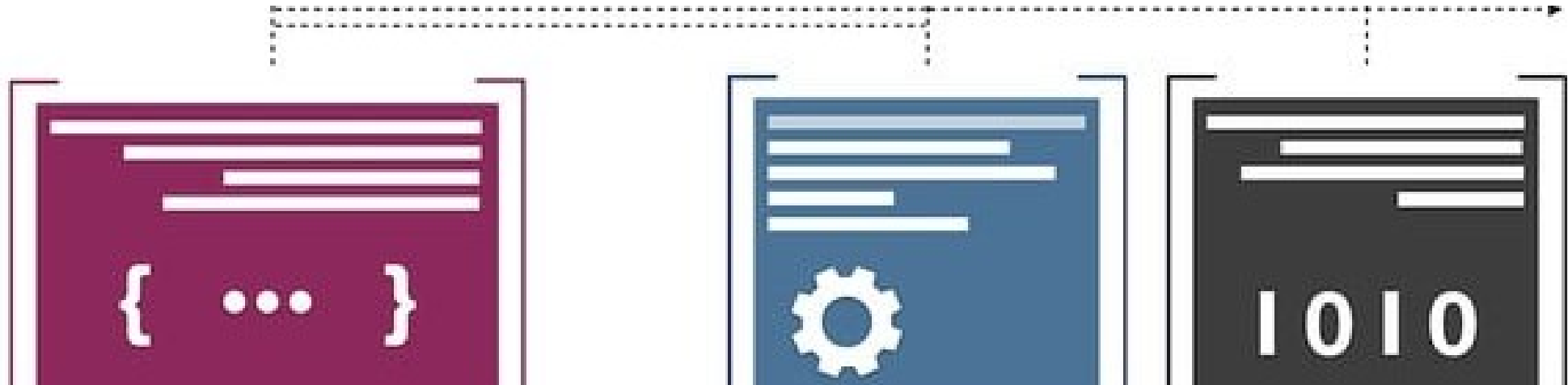
Geração de Código

Ciência da Computação | 6ª etapa

Prof. Dr. Leandro Carlos Fernandes



P-código, P-máquina & MEPA



P-Máquina

Def.: é uma **máquina virtual** hipotética, idealizada para executar um código intermediário chamado **P-Código**.

Arquitetura baseada em **Pilha** (*stack-based*).

Em vez de usar múltiplos registradores, uma P-máquina executa a maioria das suas instruções usando uma pilha de dados.

É uma arquitetura particularmente eficiente para a avaliação de expressões e chamadas de sub-rotinas em linguagens estruturadas (como Pascal, por exemplo).



Sua estrutura é bastante simples, sendo constituída pelos seguintes componentes:

Memória de Código: que armazena as instruções do programa (em P-Código) a serem executadas.

Memória de Dados (Pilha): área utilizada para o armazenamento de variáveis, resultados intermediários de expressões e dados em chamadas de sub-rotinas (parâmetros e retorno).

Registradores: que controlam o funcionamento da máquina.

- PC (*Program Counter*): que indica na memória de código qual é a próxima instrução a ser executada;
- SP (*Stack Pointer*): que aponta para o topo da pilha de dados; e
- MP (*Mark Pointer*): que contém a base do registro de ativação do procedimento atual (essencial para escopo de variáveis).



P-código

Def.: é uma **linguagem de montagem** (assembly) para a **P-Máquina**, correspondente ao código intermediário que era gerado pelo compilador Pascal nos anos 70.

Características:

Instruções simples e de baixo nível.

Projetado para ser compacto e fácil de interpretar.



As instruções do P-código compreendem, essencialmente, as seguintes categorias:

Aritméticas e Relacionais

que desempilham dois operandos, realizam a operação e empilham o resultado.

Carga e Armazenamento

que empilham constantes, valores de uma variável a partir de seu endereço de memória; ou, armazenam resultados em um endereço de memória.

Controle de Fluxo

que realizam desvios incondicionais; ou desvios condicionais se o valor no topo da pilha for falso.

Ativação de Procedimentos

instruções relacionadas ao preparo para a chamada de um novo procedimento; que realizam a chamada em si e que retornam do procedimento.



MEPA (Máquina de Execução de PAscal)

Def.: tal como a *P-Máquina*, é uma **máquina virtual baseada em pilha**, proposta pelo prof. Tomasz Kowaltowski e projetada especificamente para o ensino de compiladores.

Objetivo didático:

conjunto de instruções e arquitetura simplificados, que facilitam a compreensão e implementação da geração de código.



Região do Programa (P): contém as instruções que serão executadas pela MEPA (i.e., memória de código).

Região da Pilha ou Dados (M): armazena os valores manipulados pelas instruções da MEPA – constantes, variáveis, resultados parciais, parâmetros, etc (i.e., memória de dados).

Registradores:

$\dot{i}$: contém o endereço da próxima instrução a ser executada em P , ou seja, $P[\dot{i}]$. Inicialmente recebe valor 0.

S : indica o elemento no topo da pilha de memória, ou seja, $M[s]$.

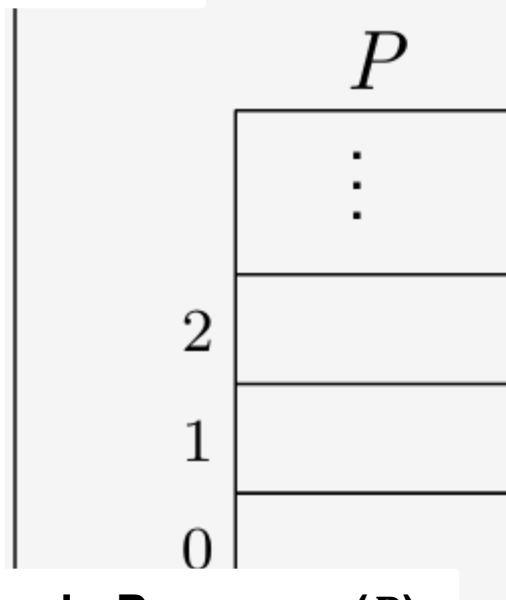
D : registradores de base (*display*) ligados a ativação atual (geralmente usado para acesso a variáveis locais).



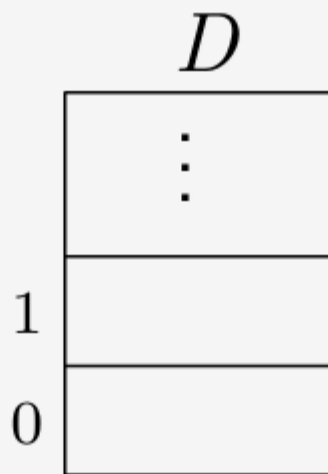
Registrador (i): indica a próxima instrução a ser executada em P , ou seja, $P[i]$.



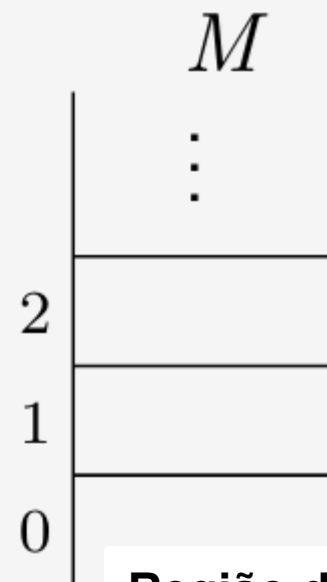
Registrador (s): indica o elemento no topo da pilha de memória, ou seja, $M[s]$.



Região do Programa (P): contém as instruções a serem executadas pela MEPA (i.e., memória de código).



Registradores de base ($display$) contexto atual (controle para acesso a variáveis locais).



Região da Pilha ou Dados (M): armazena os dados manipulados pelas instruções da MEPA (i.e., memória de dados)



Conjunto de Instruções da MEPA

As instruções da MEPA utilizam mnemônicos com quatro letras e, dependendo de sua natureza, podem ter alguns parâmetros.

Em geral, os operandos são implícitos e encontram-se armazenados na região da pilha, ou seja, em $M[]$ (memória de dados).



Aritméticas:

SOMA (Soma)	$M[s-1] \leftarrow M[s-1] + M[s]$ $s \leftarrow s - 1$	Soma dois valores no topo pilha e armazena o resultado de volta nela
SUBT (Subtração)	$M[s-1] \leftarrow M[s-1] - M[s]$ $s \leftarrow s - 1$	Subtrai dois valores no topo da pilha, guardando o resultado de volta nela.
MULT (Multiplicação)	$M[s-1] \leftarrow M[s-1] * M[s]$ $s \leftarrow s - 1$	Multiplica dois valores no topo pilha e armazena o resultado a pilha.
DIVI (Divisão)	$M[s-1] \leftarrow M[s-1] // M[s]$ $s \leftarrow s - 1$	Divide dois valores no topo da pilha e guarda o resultado de volta nela.
INVR (Inverte valor)	$M[s] \leftarrow -M[s]$	Equivale ao operador menos unário, que inverte o sinal do número.



Exemplo

Como ficaria a expressão
 $a + (b/9 - 3) * c$
em código MEPA?

1º passo: converter a expressão da notação infixa para posfixa;

2º passo: traduzir a expressão para instruções MEPA e carregá-las na memória P[].

3º passo: executar as instruções da memória P[] utilizando M[] como memória auxiliar.

Supondo que as variáveis tenham sido alocadas nos endereços a:100, b:102 e c:99, teríamos:

```
CRVL 0,100 ; empilha a
CRVL 0,102 ; empilha b
CRCT 9      ; empilha 9
DIVI        ; b/9
CRCT 3      ; empilha 3
SUBT        ; resultado - 3
CRVL 0,99   ; empilha c
MULT        ; resultado * c
SOMA        ; a + resultado
```



Essenciais:

INPP (Iniciar Prog Principal)	$s \leftarrow -1$ $D[0] \leftarrow 0$	Inicia a execução do programa, inicializando a pilha.
PARA		Termina a execução do programa.

Entrada e Saída:

LEIT (Leitura)	$s \leftarrow s + 1$ $M[s] \leftarrow \text{próxima entrada}$	Lê um valor da entrada padrão e empilha.
IMPR (Imprime)	$\text{imprime}(M[s])$ $s \leftarrow s - 1$	Desempilha um valor e o imprime na saída padrão.



Carga e Armazenamento:

AMEM m (Aloca Memória)	$s \leftarrow s + m$	Aloca m espaços na pilha para variáveis locais.
DMEM m (Desaloca Memória)	$s \leftarrow s - m$	Desaloca m espaços da pilha, liberando recursos locais.
CRCT k (Carrega Constante)	$s \leftarrow s + 1$ $M[s] \leftarrow k$	Empilha a constante k
CRVL m, n (Carrega Valor)	$s \leftarrow s + 1$ $M[s] \leftarrow M[D[m] + n]$	Empilha o valor da variável de endereço n (do nível m).
ARMZ m, n (Armazena Valor)	$M[D[m] + n] \leftarrow M[s]$ $s \leftarrow s - 1$	Armazena na variável de endereço n (do nível m) o valor desempilhado.



Exemplo

Qual seria o código MEPA gerado para o programa SAL dado abaixo?

```
module Ex1;
globals
  a,b,c: int;
proc main()
start
  scan(b);
  scan(c);
  a:=b+c*2;
  print(a);
end
```

```
INPP          ; module Ex1
AMEM 3        ; globals a,b,c
LEIT
ARMZ 0,1      ; scan(b)
LEIT
ARMZ 0,2      ; scan(c)
CRVL 0,1      ; empilha 'b'
CRVL 0,2      ; empilha 'c'
CRCT 2        ; empilha 2
MULT          ; c * 2
SOMA          ; b + resultado
ARMZ 0,0      ; a := resultado
CRVL 0,0      ; empilha 'a'
IMPR          ; imprime a
DMEM 3
PARA          ; end
```



Relacionais:

CMME (Menor)	se $M[s-1] < M[s]$ então $M[s-1] \leftarrow 1$ senão $M[s-1] \leftarrow 0$ $s \leftarrow s - 1$	CMEG (Menor ou Igual)	se $M[s-1] \leq M[s]$ então $M[s-1] \leftarrow 1$ senão $M[s-1] \leftarrow 0$ $s \leftarrow s - 1$
CMMA (Maior)	se $M[s-1] > M[s]$ então $M[s-1] \leftarrow 1$ senão $M[s-1] \leftarrow 0$ $s \leftarrow s - 1$	CMAG (Maior ou igual)	se $M[s-1] \geq M[s]$ então $M[s-1] \leftarrow 1$ senão $M[s-1] \leftarrow 0$ $s \leftarrow s - 1$
CMIG (Igual)	se $M[s-1] = M[s]$ então $M[s-1] \leftarrow 1$ senão $M[s-1] \leftarrow 0$ $s \leftarrow s - 1$	CMDG (Diferente)	se $M[s-1] \neq M[s]$ então $M[s-1] \leftarrow 1$ senão $M[s-1] \leftarrow 0$ $s \leftarrow s - 1$

Compara os dois valores no topo pilha (menor, maior, igual, etc.) e empilha o resultado booleano.



Lógicos:

CONJ (Conjunção)	$M[s-1] \leftarrow M[s-1] \text{ and } M[s]$ $s \leftarrow s - 1$	
DISJ (Disjunção)	$M[s-1] \leftarrow M[s-1] \text{ or } M[s]$ $s \leftarrow s - 1$	
NEGA (Negação)	$M[s] \leftarrow \text{not } M[s]$ $s \leftarrow s - 1$	

Executa a operação lógica sobre valores booleanos que estão no topo pilha, empilhando um resultado também booleano.



Desvios e Rótulos

DSVS p (Desv. Sempre)	$i \leftarrow p$	Salta incondicionalmente para a instrução com rótulo p .
DSVF p (Desv. se Falso)	$\begin{array}{l} \text{se } M[s] = 0 \\ \quad \text{então } i \leftarrow p \\ \quad \text{senão } i \leftarrow i + 1 \\ s \leftarrow s - 1 \end{array}$	Caso a última operação resulte em 0, salta para a instrução com rótulo p , senão executa a próxima instrução.
NADA (Sem operação)	$i \leftarrow i + 1$	Simplesmente segue para a próxima instrução.

Utilizaremos rótulos simbólicos ao invés de endereços de programa, pois o interpretador da MEPA aceita rótulos.



A partir dessas instruções é possível estabelecer o que chamamos de “gabaritos de tradução” para comandos condicionais e iterativos de uma linguagem (como C, por exemplo).

if (*expr*) *cmd1* **else** *cmd2*:

```
    ...           ; expr
    DSVF L1
    ...           ; cmd1
    DSVS L2
L1: NADA
    ...           ; cmd2
L2: NADA
```

while (*expr*) *cmd*:

```
L1: NADA           ; while
    ...           ; expr
    DSVF L2
    ...           ; cmd
    DSVS L1
L2: NADA
```



Considere o fragmento de código a seguir:

```
while( $s \leq n$ )  
     $s = s + 3$ ;
```

Com base no gabarito de tradução, o código MEPA gerado para ele seria:

```
L1: NADA           ; while  
    CRVL  $f, s$   
    CRVL  $f, n$   
    CMEG           ;  $s \leq n$   
    DSVF L2  
    CRVL  $f, s$   
    CRCT 3  
    SOMA  
    ARMZ  $f, s$      ;  $s = s + 3$   
    DSVS L1  
L2: NADA           ; fim do loop
```



Exemplo

Como seria a tradução de comandos condicionais aninhados, como o que é dado no fragmento abaixo?

```
if (a>b) {  
    q = p && q;  
} else {  
    if (a<2*b)  
        p := 1; //true  
    else  
        q := 0; //false  
}
```

CRVL	f, a	CRVL	f, b
CRVL	f, b	MULT	
CMMA		CMME	
DSVF	L3	DSVF	L5
CRVL	f, p	CRCT	1
CRVL	f, q	ARMZ	f, p
CONJ		DSVS	L6
ARMZ	f, q	L5:	NADA
DSVS	L4	CRCT	0
L3:	NADA	ARMZ	f, q
CRVL	f, a	L6:	NADA
CRCT	2	L4:	NADA

Usamos rótulos simbólicos ao invés dos endereços das variáveis para facilitar o entendimento



Exercício Resolvido



Reconhecendo que a tradução do programa SAL abaixo equivale ao código MEPA dado lado, simule sua execução na máquina virtual.

```
module Ex1;
globals
  a,b,c: int;
proc main()
start
  scan(b);
  scan(c);
  a:=b+c*2;
  print(a);
end
```

```
INPP           ; module Ex1
AMEM 3         ; globals a,b,c
LEIT
ARMZ 0,1       ; scan(b)
LEIT
ARMZ 0,2       ; scan(c)
CRVL 0,1       ; empilha 'b'
CRVL 0,2       ; empilha 'c'
CRCT 2         ; empilha 2
MULT           ; c * 2
SOMA           ; b + resultado
ARMZ 0,0       ; a := resultado
CRVL 0,0       ; empilha 'a'
IMPR           ; imprime a
DMEM 3
PARA           ; end
FIM
```

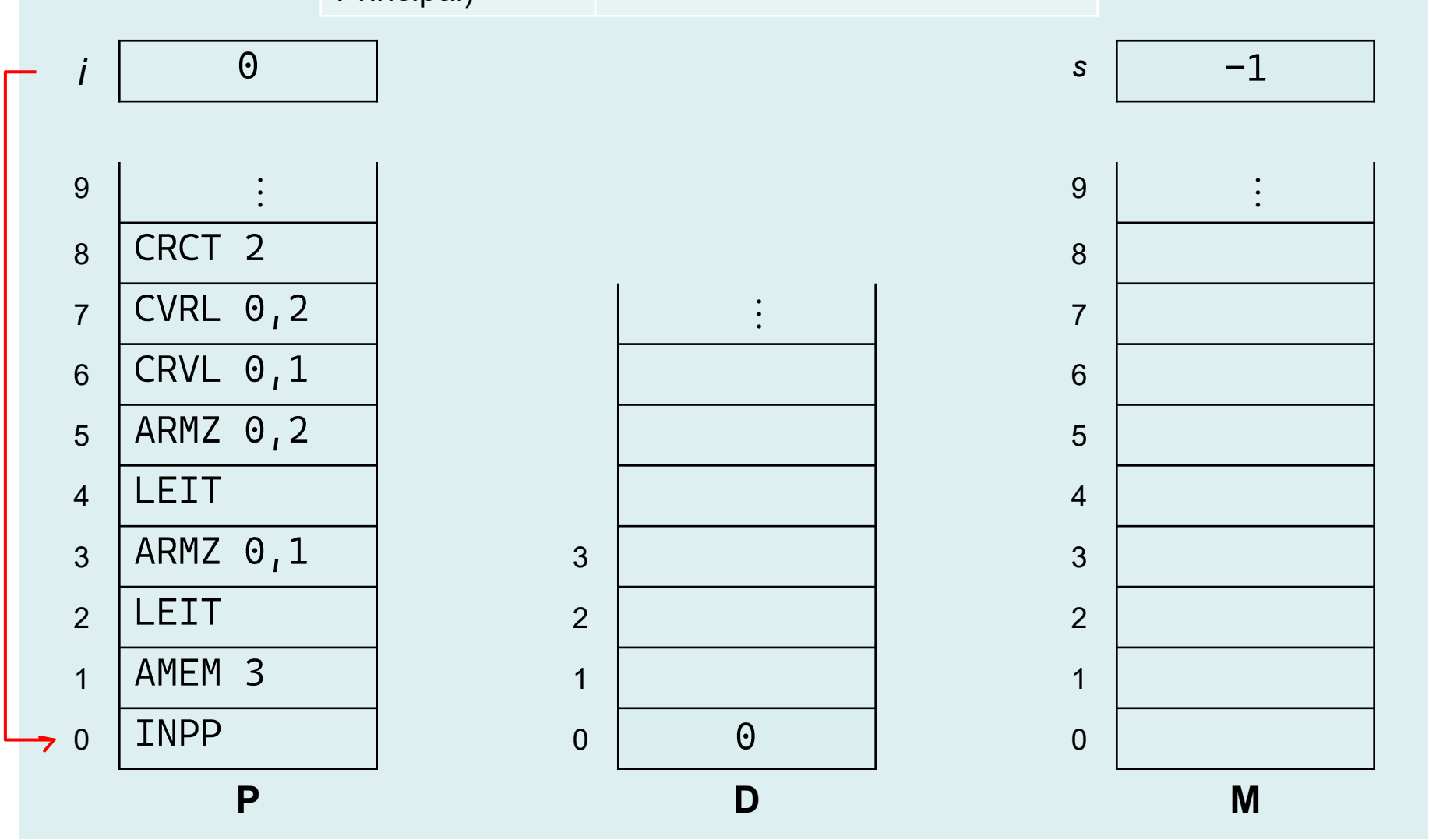


Exercício resolvido

INPP

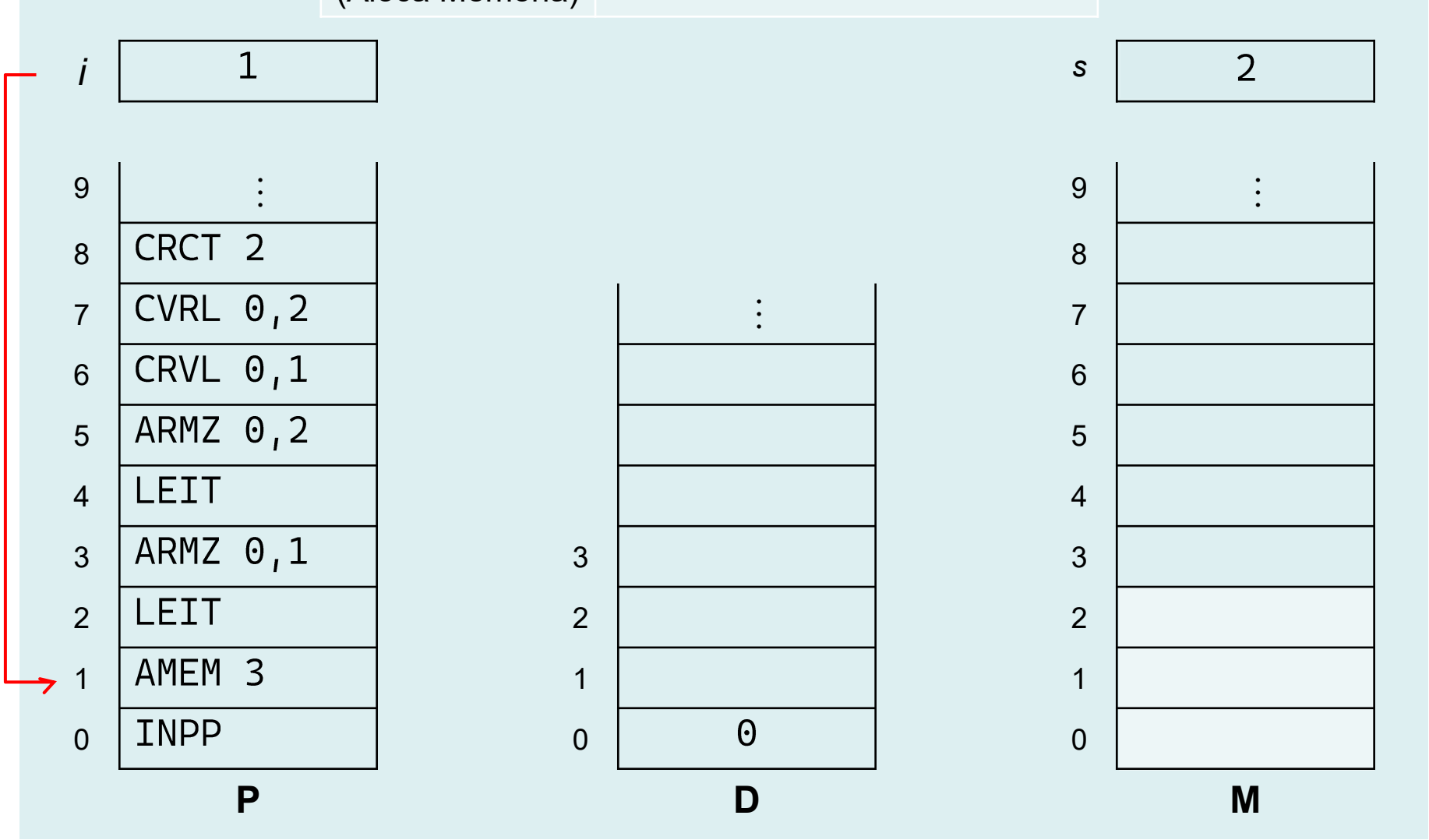
(Iniciar Prog
Principal)

$s \leftarrow -1$
 $D[0] \leftarrow 0$



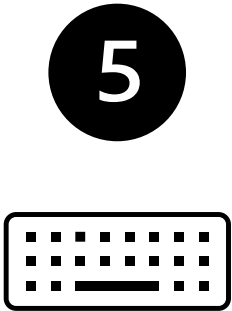
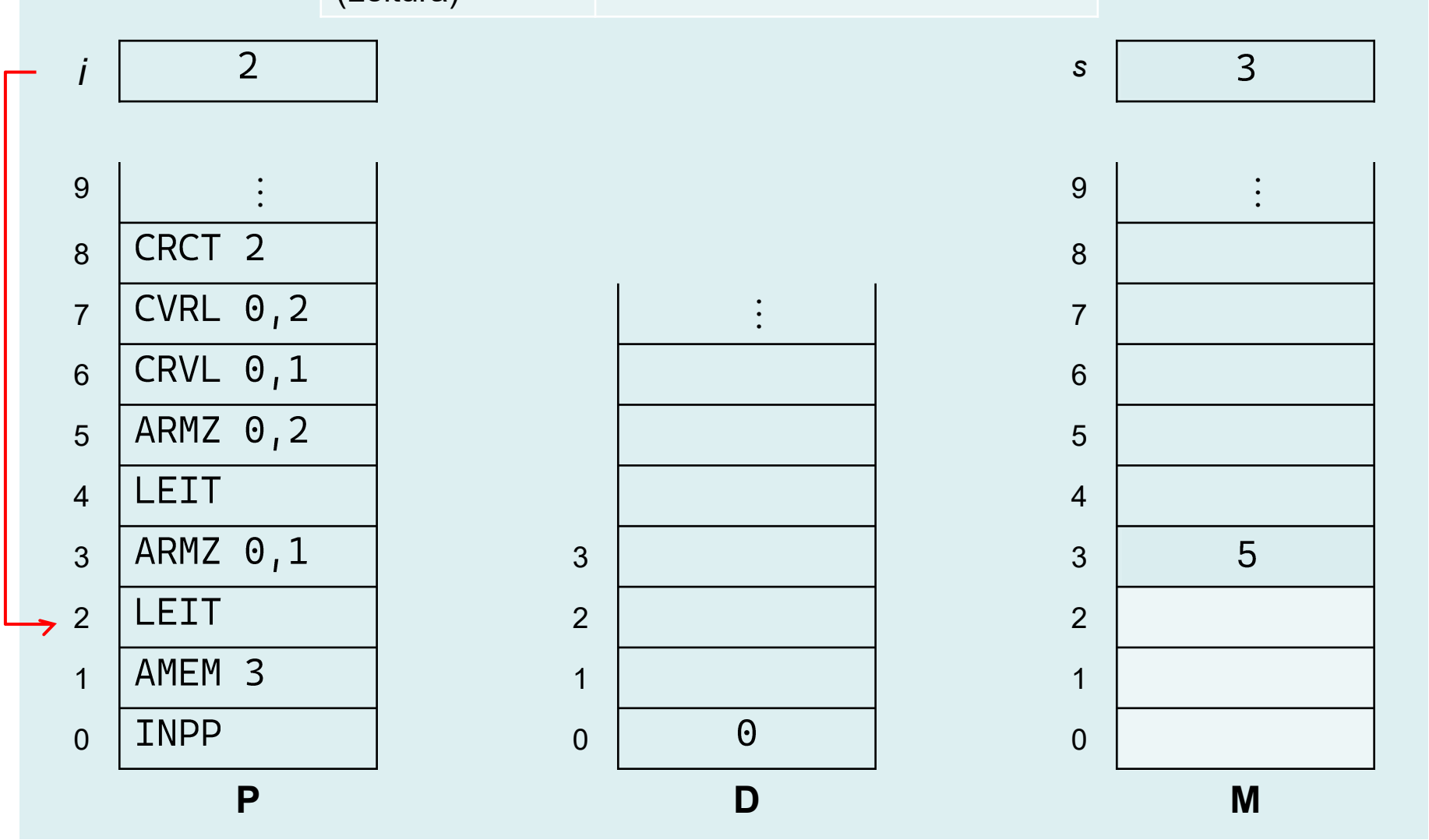
AMEM *m*
(Aloca Memória)

$$s \leftarrow s + m$$



LEIT
(Leitura)

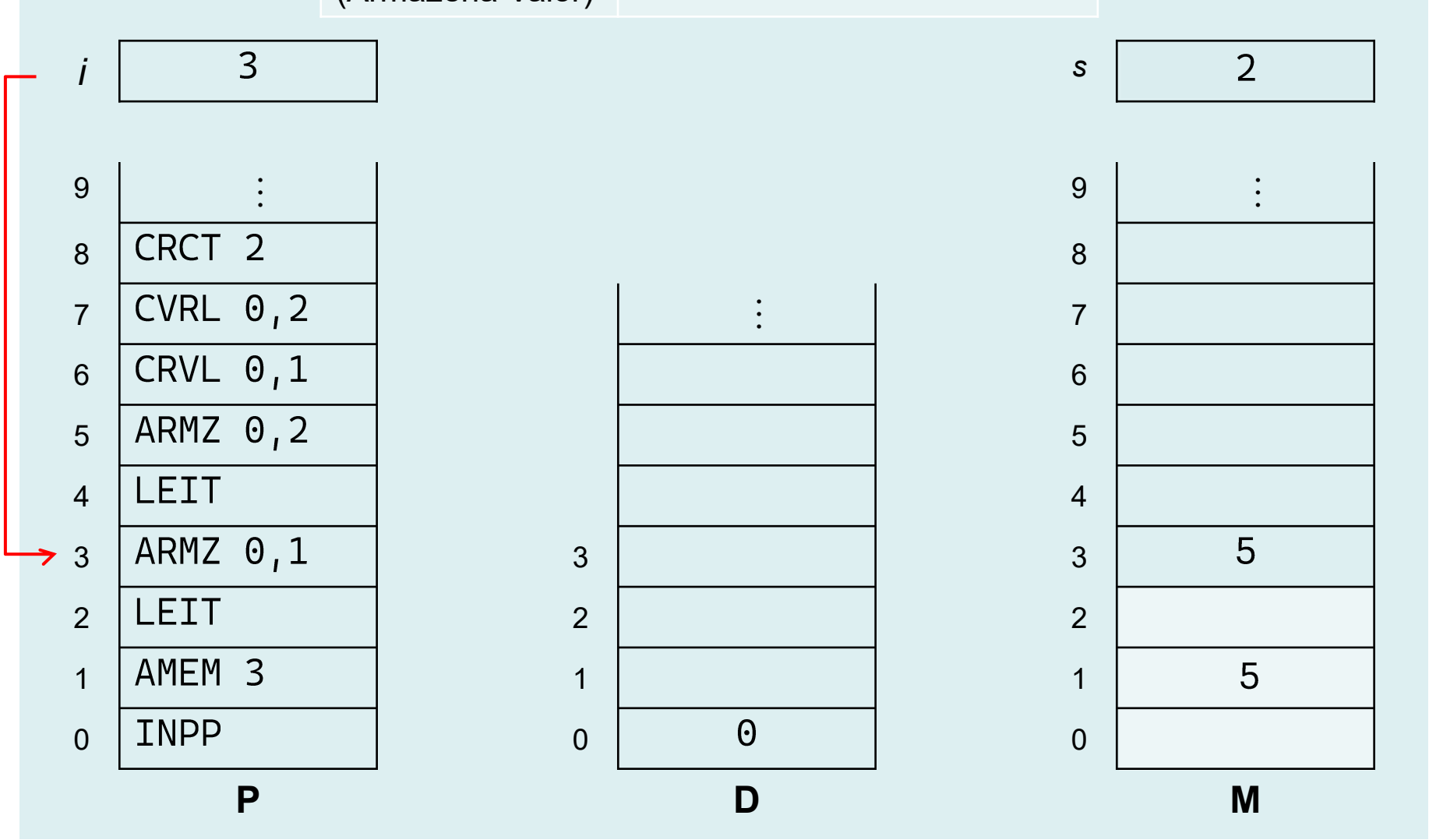
$s \leftarrow s + 1$
 $M[s] \leftarrow \text{entrada}$



Exercício resolvido

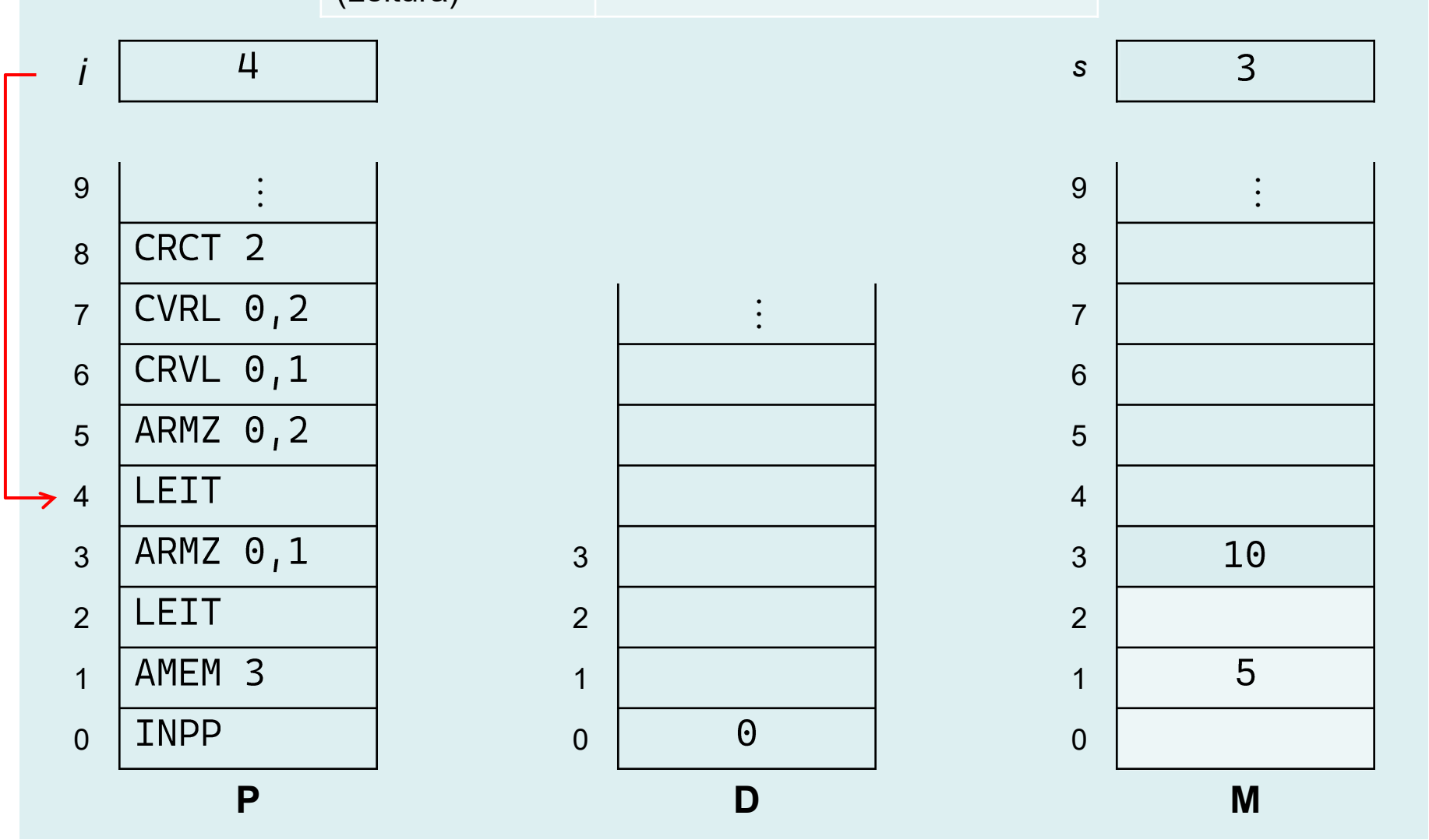
ARMZ m, n
(Armazena Valor)

$M[D[m] + n] \leftarrow M[s]$
 $s \leftarrow s - 1$



LEIT
(Leitura)

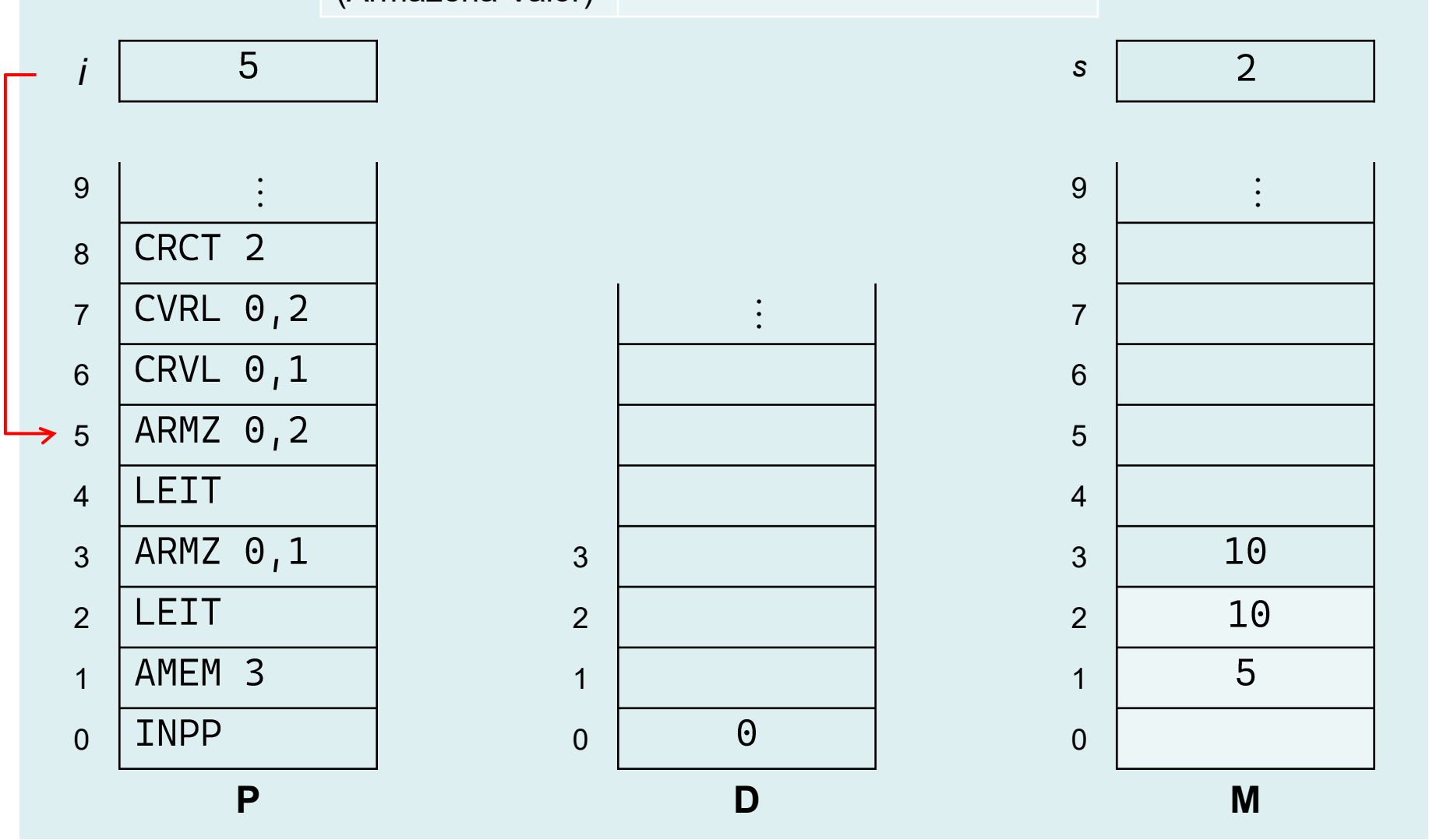
$s \leftarrow s + 1$
 $M[s] \leftarrow \text{entrada}$



Exercício resolvido

ARMZ m, n
(Armazena Valor)

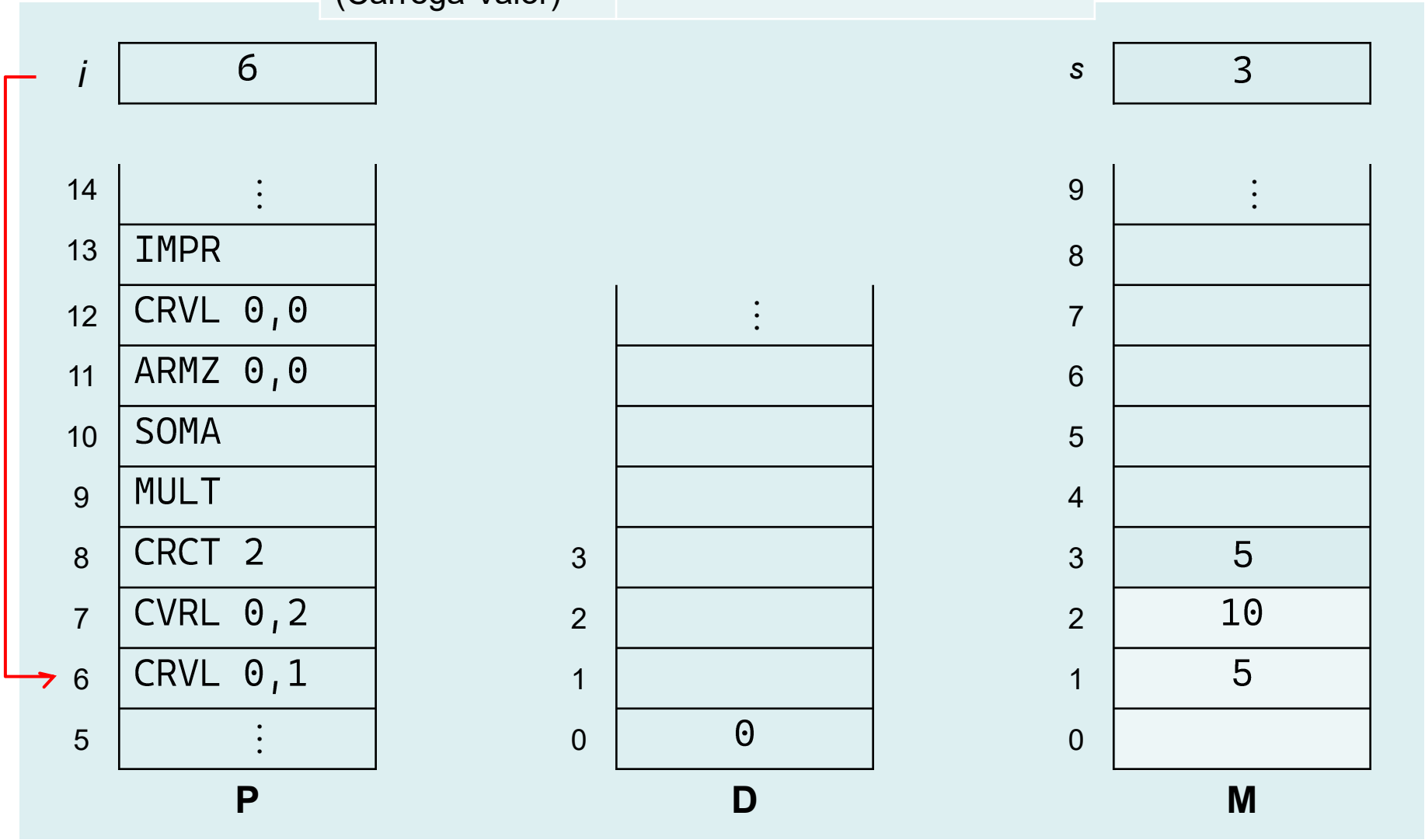
$M[D[m] + n] \leftarrow M[s]$
 $s \leftarrow s - 1$



Exercício resolvido

CRVL m, n
(Carrega Valor)

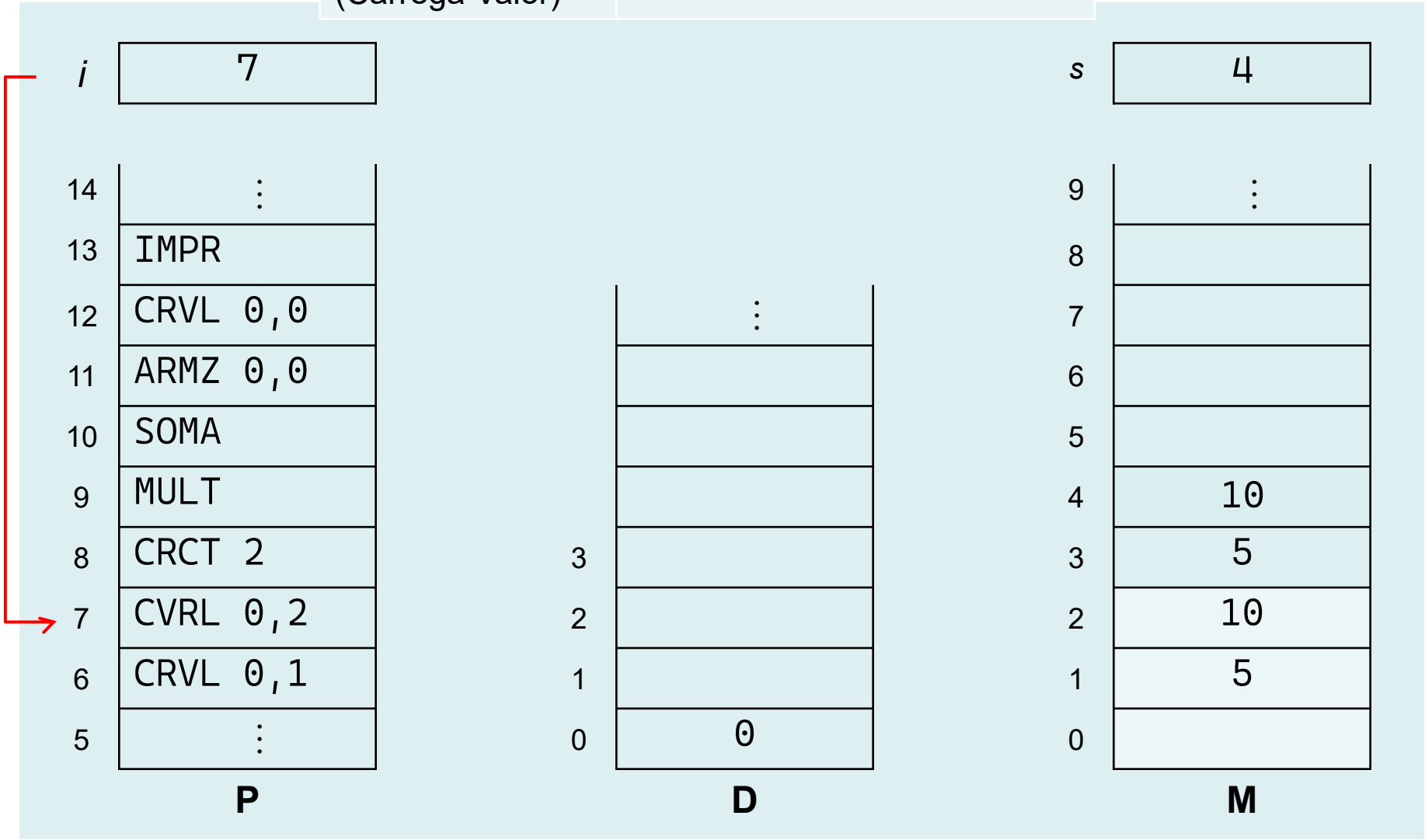
$s \leftarrow s + 1$
 $M[s] \leftarrow M[D[m] + n]$



Exercício resolvido

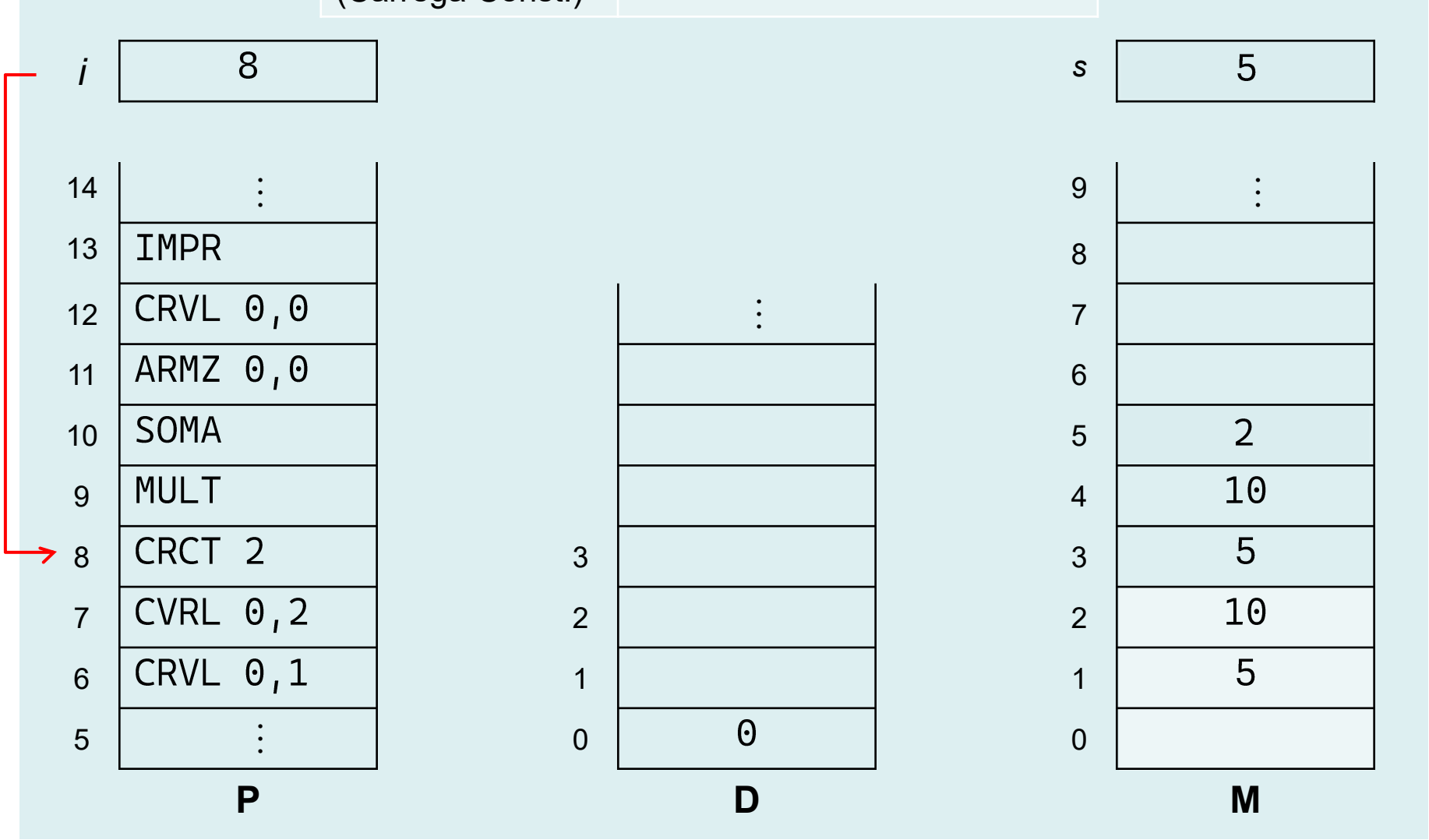
CRVL m, n
(Carrega Valor)

$s \leftarrow s + 1$
 $M[s] \leftarrow M[D[m] + n]$



CRCT *k*
(Carrega Const.)

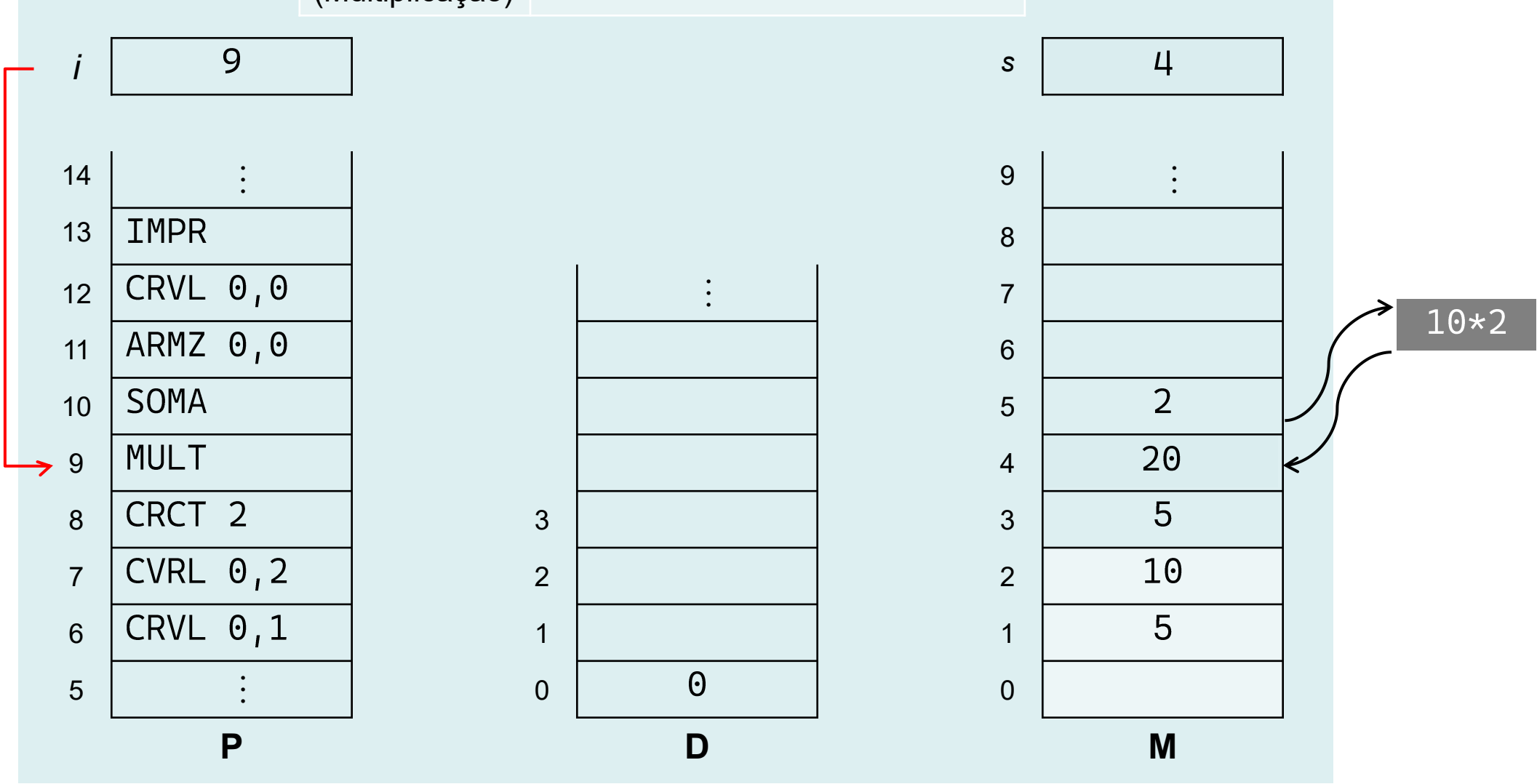
$s \leftarrow s + 1$
 $M[s] \leftarrow k$



Exercício resolvido

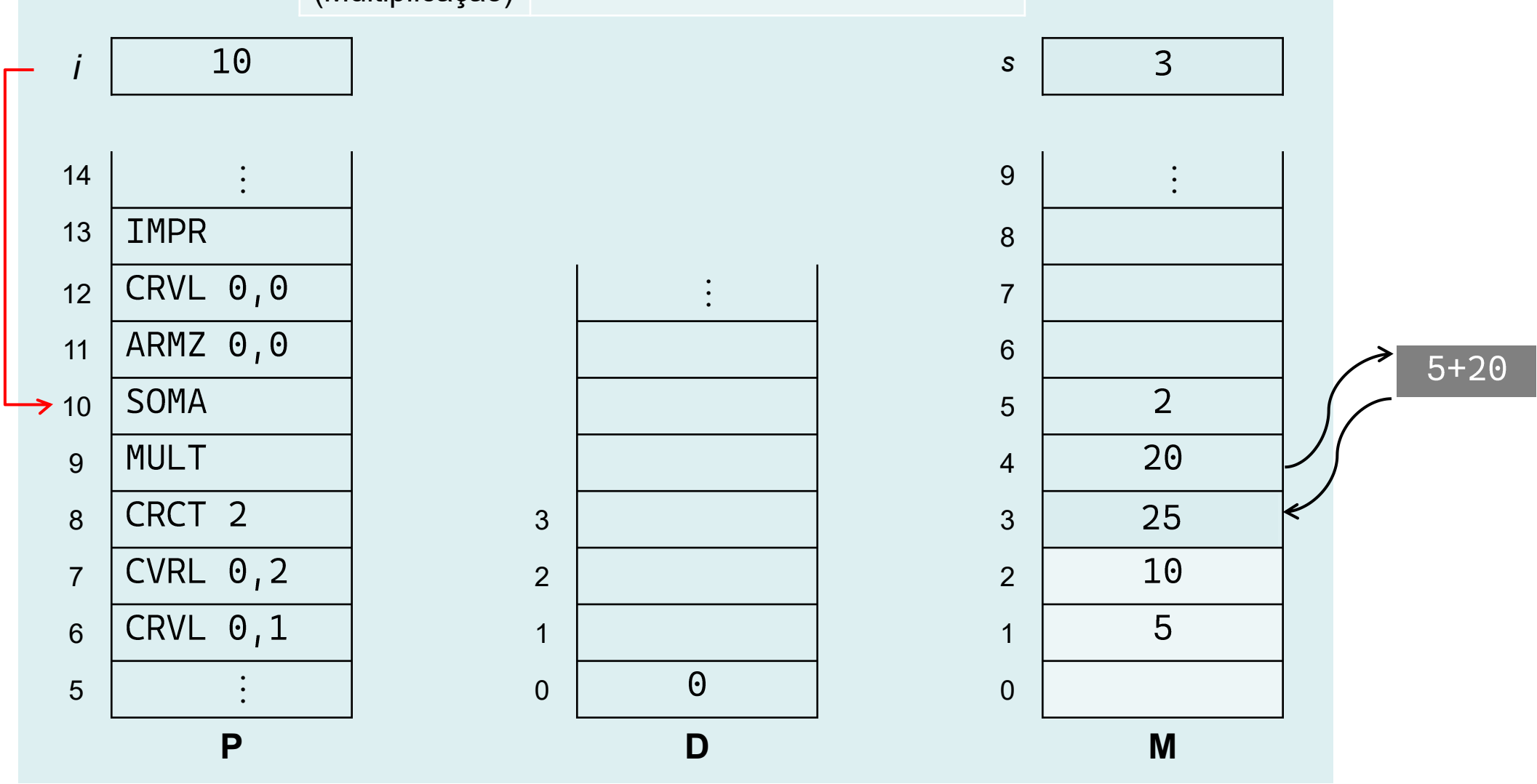
MULT
(Multiplicação)

$M[s-1] \leftarrow M[s-1] * M[s]$
 $s \leftarrow s - 1$



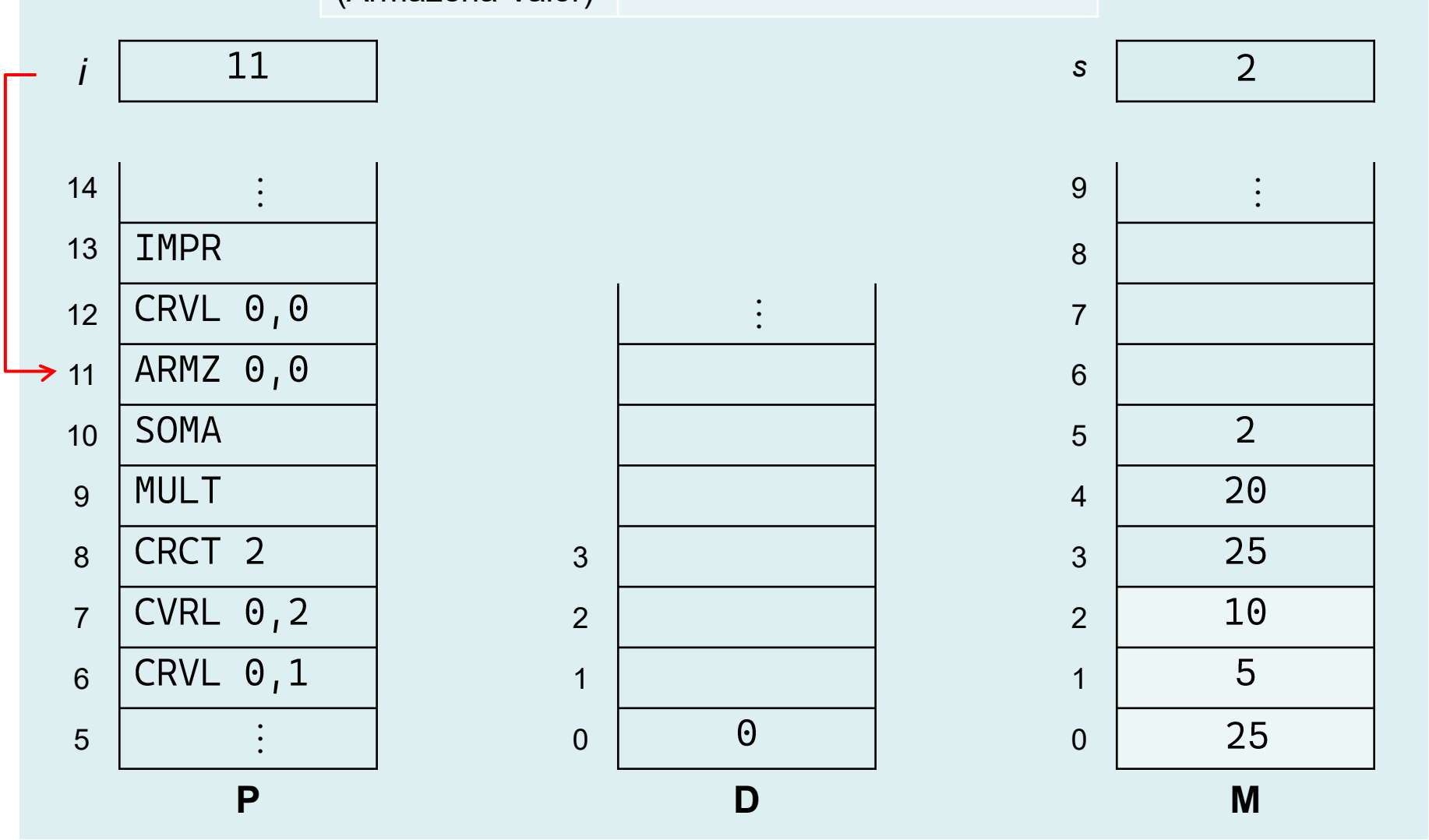
MULT
(Multiplicação)

$M[s-1] \leftarrow M[s-1] * M[s]$
 $s \leftarrow s - 1$



ARMZ m, n
(Armazena Valor)

$M[D[m] + n] \leftarrow M[s]$
 $s \leftarrow s - 1$



Considere a expressão a seguir, que é dada em uma ling. de alto nível (como C) e, portanto, codificada em notação infixa (i.e., operando operador operando):

$$a = (b < c) \&\& (b > 0);$$

Suponha que o compilador tenha alocado as variáveis a , b e c nos seguintes endereços de M :

$$a:00 \quad ; \quad b:01 \quad \text{e} \quad c:02$$

Qual seria o estado da memória após a execução desse comando, sabendo que o registrador S aponta para o endereço 06 da memória $M[]$?





Universidade Presbiteriana
Mackenzie



Faculdade de
Computação e Informática

